In [3]:
```
import pandas as pd
```

01 CFPB Stratified Sampling and Feature Preparation

Objective: prepare the CFPB complaints data for the credit card analysis workflow by engineering time and geography features, reviewing product-level complaint structure, and exporting a trimmed credit card subset for downstream EDA and forecasting notebooks.

This notebook prepares the following artifacts:

- engineered temporal and geographic features (`year_quarter` , `region` , `stratum`) on the raw complaints data
- exploratory product, time, and region diagnostics used to justify narrowing the project scope to credit card complaints
- final trimmed credit card subset exported for notebook 02 EDA and the notebook 03 baseline forecasting workflows

In [4]:
```
# Load the CFPB complaints CSV into a DataFrame
df = pd.read_csv(r'..\data\complaints.csv')
```

In [6]:
```
# Preview the dataset structure and sample records
df.head()
```

Out[6]:

	Date received	Product	Sub-product	Issue	Sub-issue	Consumer complaint narrative	Company public response	Com
0	2020-07-13	Mortgage	FHA mortgage	Trouble during payment process	NaN	I currently have my mortgage with GMFS, servic...	NaN	S Honebee
1	2023-12-16	Credit reporting or other personal consumer re...	Credit reporting	Incorrect information on your report	Information belongs to someone else	NaN	NaN	Factor
2	2024-01-27	Credit reporting or other personal consumer re...	Credit reporting	Incorrect information on your report	Account information incorrect	NaN	Company has responded to the consumer and the ...	TRANSUN INTERMEI HOLDI
3	2019-05-22	Credit reporting, credit repair services, or o...	Credit reporting	Problem with fraud alerts or security freezes	NaN	Have had a couple years of problems getting an...	Company has responded to the consumer and the ...	Exp Inform Solution
4	2023-12-21	Credit reporting or other personal consumer re...	Credit reporting	Incorrect information on your report	Personal information incorrect	NaN	Company has responded to the consumer and the ...	Exp Inform Solution

In [7]:
```
# Check total row count in the raw dataset
print(f"{len(df):,}")
```

```
13,843,560
```

In [8]:
```
# Inspect complaint volume by product
df['Product'].value_counts().reset_index(name='num_instances')
```

Out[8]:

	Product	num_instances
0	Credit reporting or other personal consumer re...	8648731
1	Credit reporting, credit repair services, or o...	2163844
2	Debt collection	1013429
3	Mortgage	440724
4	Checking or savings account	344990
5	Credit card	290278
6	Credit card or prepaid card	206368
7	Money transfer, virtual currency, or money ser...	169517
8	Credit reporting	140429
9	Student loan	121694
10	Vehicle loan or lease	88829
11	Bank account or service	86204
12	Consumer Loan	31574
13	Payday loan, title loan, or personal loan	30640
14	Payday loan, title loan, personal loan, or adv...	26785
15	Prepaid card	19736
16	Debt or credit management	7817
17	Payday loan	5541
18	Money transfers	5354
19	Other financial service	1058
20	Virtual currency	18

In [9]:
```
# Inspect complaint volume by sub-product
df['Sub-product'].value_counts().reset_index(name='num_instances')
```

Out[9]:

	Sub-product	num_instances
0	Credit reporting	10755952
1	I do not know	372881
2	General-purpose credit card or charge card	338623
3	Checking account	337944
4	Other debt	181044
...	...	...
81	Traveler's/Cashier's checks	88
82	Student prepaid card	55
83	Tax refund anticipation loan or check	42
84	Transit card	37
85	Electronic Benefit Transfer / EBT card	12

86 rows × 2 columns

In [10]:
```
# Verify min/max complaint received dates
print(f"Dates range from {min(df['Date received'])} to {max(df['Date received'])}")
```

```
Dates range from 2011-12-01 to 2026-02-24
```

Step 1: Creating Year-Quarter

In [5]:
```
df['Date received'] = pd.to_datetime(df['Date received'], errors='coerce')

df['year_quarter'] = (
    df['Date received']
      .dt.to_period('Q')
      .astype(str)
)
```

This gives values like '2021Q1', '2022Q3', etc.

Step 2: Define Initial Geography Field

Before moving to regional aggregation, a state-level geography field is created to support exploratory strata design and sparsity checks.

In [6]:
```
df['geo'] = df['State']
```

Step 3: Mapping State to Region

For the final workflow, I use U.S. Census Bureau regions instead of individual states. This reduces sparsity while still preserving meaningful geographic structure for the downstream sample.

1. Northeast

- CT, ME, MA, NH, RI, VT
- NJ, NY, PA

2. Midwest

- IL, IN, MI, OH, WI
- IA, KS, MN, MO, NE, ND, SD

3. South

- DE, FL, GA, MD, NC, SC, VA, DC, WV
- AL, KY, MS, TN
- AR, LA, OK, TX

4. West

- AZ, CO, ID, MT, NV, NM, UT, WY
- AK, CA, HI, OR, WA

5. Other

- Any remaining values in State (e.g., territories or non-state codes)

In [7]:
```
northeast = ['CT','ME','MA','NH','RI','VT','NJ','NY','PA']
midwest   = ['IL','IN','MI','OH','WI','IA','KS','MN','MO','NE','ND','SD']
south     = ['DE','FL','GA','MD','NC','SC','VA','DC','WV',
             'AL','KY','MS','TN','AR','LA','OK','TX']
west      = ['AZ','CO','ID','MT','NV','NM','UT','WY','AK','CA','HI','OR','WA']

def map_region(state):
    if state in northeast:
        return 'Northeast'
    elif state in midwest:
        return 'Midwest'
    elif state in south:
        return 'South'
    elif state in west:
        return 'West'
    else:
        return 'Other'

df['region'] = df['State'].apply(map_region)
```

Rows with unresolved geography are removed so each record can be assigned to a valid regional stratum in later steps.

In [14]:
```
# Drop rows without valid region
df = df.dropna(subset=['region'])
```

Regional share is reviewed here to confirm that the aggregated geography feature is coherent and suitable for proportional sampling.

In [15]:
```
df['region'].value_counts(normalize=True)
```

Out[15]:
```
region
South        0.540400
West         0.161662
Northeast    0.158621
Midwest      0.130691
Other        0.008627
Name: proportion, dtype: float64
```

Step 4: Finalize Geography for Feature Engineering

The region field is retained as the main geography feature because it is interpretable, stable over time, and appropriate for the downstream credit card subset analysis.

In [9]:
```
df['year_quarter'] = pd.to_datetime(df['Date received']).dt.to_period('Q').astype(s

df['stratum'] = (
    df['Product'].astype(str) + "|" +
    df['year_quarter'].astype(str) + "|" +
    df['region'].astype(str)
)
```

Step 5: Sanity Check Stratum Granularity

A quick cardinality check confirms that the product-time-region strata are informative without becoming so sparse that they stop being useful for exploratory diagnostics and downstream subset design.

In [17]:
```
num_unique_strata = df['stratum'].nunique()
print(f"Total number of unique strata: {num_unique_strata:,}")
```

```
Total number of unique strata: 2,752
```

In [18]:
```
strata_counts = df.groupby(['Product', 'year_quarter', 'region']).size()

strata_counts.describe()
```

Out[18]:
```
count      2752.000000
mean       5030.363372
std       35783.133537
min           1.000000
25%         142.750000
50%         539.000000
75%        1692.250000
max      917786.000000
dtype: float64
```

Interpretation:

- The median stratum size is large enough to preserve meaningful product-time-region structure.
- The distribution remains right-skewed, which is expected in CFPB data because complaint activity is concentrated in a subset of product-time-region combinations.
- A limited long tail of small strata is acceptable for descriptive diagnostics, especially since the downstream workflow ultimately narrows to the credit card subset.

In [19]:
```
print(f"Number of strata with fewer than 20 samples: {(strata_counts < 20).sum():,}
```

```
Number of strata with fewer than 20 samples: 194 out of 2,752 (i.e., 0.07%).
```

Conclusion from diagnostics: the current product-time-region structure is coherent enough for feature preparation and for motivating the narrower credit card analysis track.

Product-Level Complaint Review

This section uses the full complaint file only to understand how complaint volume evolves across products before the workflow is narrowed to the credit card subset.

In [20]:
```
# Generate total number of complaints for each quarter
quarterly_complaints = df.groupby('year_quarter').size().reset_index(name='num_comp
quarterly_complaints
```

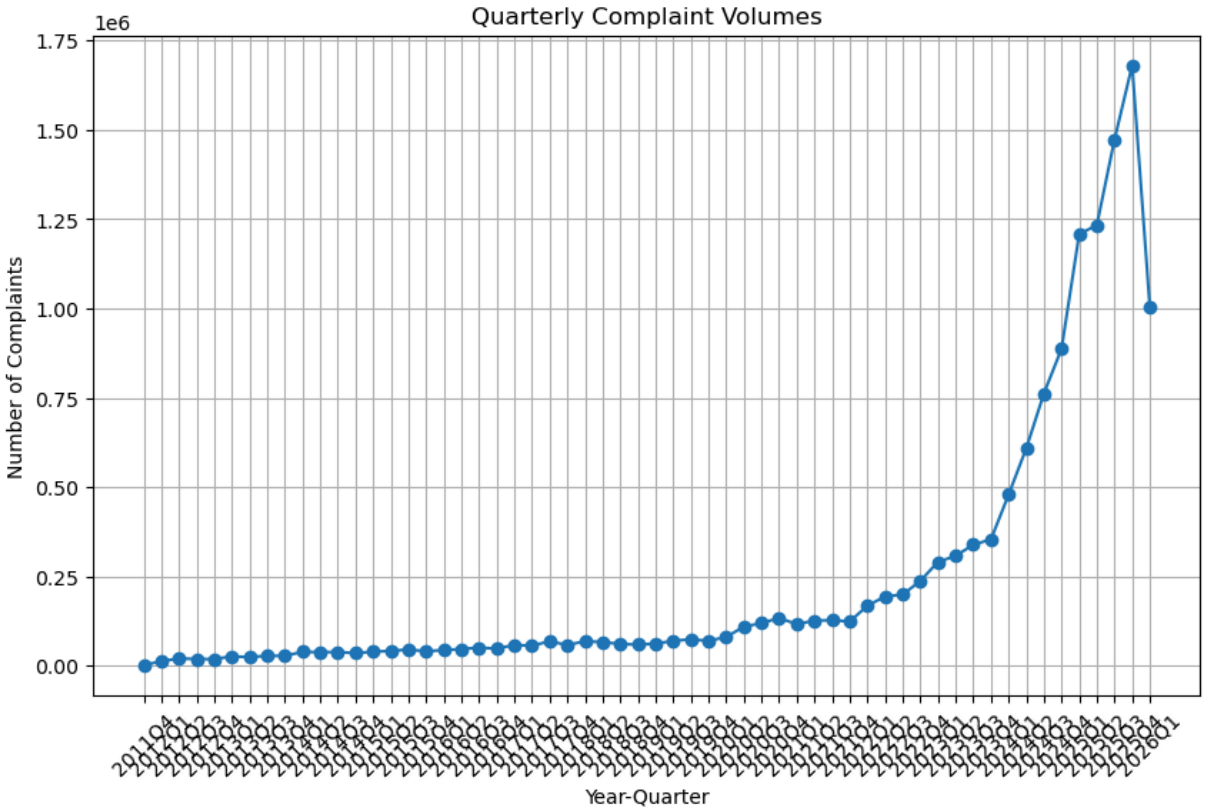
Out[20]:

	year_quarter	num_complaints
0	2011Q4	2536
1	2012Q1	12969
2	2012Q2	21160
3	2012Q3	19125
4	2012Q4	19118
5	2013Q1	26874
6	2013Q2	24837
7	2013Q3	28471
8	2013Q4	28033
9	2014Q1	39607
10	2014Q2	38513
11	2014Q3	39049
12	2014Q4	35832
13	2015Q1	39812
14	2015Q2	41940
15	2015Q3	46008
16	2015Q4	40669
17	2016Q1	44585
18	2016Q2	47178
19	2016Q3	51296
20	2016Q4	48342
21	2017Q1	58849
22	2017Q2	56370
23	2017Q3	69162
24	2017Q4	58459
25	2018Q1	69234
26	2018Q2	66746
27	2018Q3	61735
28	2018Q4	59483
29	2019Q1	62525

	year_quarter	num_complaints
30	2019Q2	70099
31	2019Q3	74695
32	2019Q4	69968
33	2020Q1	80969
34	2020Q2	109686
35	2020Q3	119724
36	2020Q4	133905
37	2021Q1	116928
38	2021Q2	126716
39	2021Q3	128041
40	2021Q4	124300
41	2022Q1	168328
42	2022Q2	194518
43	2022Q3	199503
44	2022Q4	237984
45	2023Q1	291118
46	2023Q2	307730
47	2023Q3	339375
48	2023Q4	353871
49	2024Q1	479266
50	2024Q2	608617
51	2024Q3	762375
52	2024Q4	887410
53	2025Q1	1208044
54	2025Q2	1233564
55	2025Q3	1472778
56	2025Q4	1680180
57	2026Q1	1005351

In [21]:
```
# Plot the quarterly complaint volumes
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
```

```
plt.plot(quarterly_complaints['year_quarter'], quarterly_complaints['num_complaints
plt.title('Quarterly Complaint Volumes')
plt.xlabel('Year-Quarter')
plt.ylabel('Number of Complaints')
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```



In [24]:
```
# Plot the distribution of complaints by product per quarter
product_counts_qtr = df.groupby(['year_quarter', 'Product']).size().reset_index(nam
```

In [31]:
```
# Tabulate the complaint counts by product and quarter (year_quarter in col. 1, and
product_counts_pivot = product_counts_qtr.pivot(index='year_quarter', columns='Prod
product_counts_pivot
```

Out[31]:

Product	Bank account or service	Checking or savings account	Consumer Loan	Credit card	Credit card or prepaid card	Credit reporting	Credit reporting or other personal consumer reports	C repor c re serv or o pers consu rep
year_quarter								
2011Q4	0.0	0.0	0.0	1260.0	0.0	0.0	0.0	
2012Q1	1191.0	0.0	141.0	3757.0	0.0	0.0	0.0	
2012Q2	4318.0	0.0	650.0	4340.0	0.0	0.0	0.0	
2012Q3	3643.0	0.0	591.0	3780.0	0.0	0.0	0.0	
2012Q4	3060.0	0.0	604.0	3476.0	0.0	1873.0	0.0	
2013Q1	3532.0	0.0	698.0	3584.0	0.0	3074.0	0.0	
2013Q2	3077.0	0.0	771.0	3310.0	0.0	3470.0	0.0	
2013Q3	3428.0	0.0	815.0	3197.0	0.0	3949.0	0.0	
2013Q4	3351.0	0.0	833.0	3014.0	0.0	3887.0	0.0	
2014Q1	3723.0	0.0	1063.0	3690.0	0.0	7484.0	0.0	
2014Q2	3736.0	0.0	1059.0	3468.0	0.0	7314.0	0.0	
2014Q3	3708.0	0.0	1652.0	3561.0	0.0	7809.0	0.0	
2014Q4	3495.0	0.0	1682.0	3255.0	0.0	6631.0	0.0	
2015Q1	3684.0	0.0	1833.0	3995.0	0.0	7885.0	0.0	
2015Q2	4142.0	0.0	1773.0	4304.0	0.0	8053.0	0.0	
2015Q3	4547.0	0.0	2332.0	4569.0	0.0	10005.0	0.0	
2015Q4	4767.0	0.0	1942.0	4432.0	0.0	8329.0	0.0	
2016Q1	4590.0	0.0	2209.0	4754.0	0.0	9839.0	0.0	
2016Q2	5072.0	0.0	2339.0	4723.0	0.0	11492.0	0.0	
2016Q3	6273.0	0.0	2502.0	5969.0	0.0	11963.0	0.0	
2016Q4	5912.0	0.0	2541.0	5618.0	0.0	10787.0	0.0	
2017Q1	5719.0	0.0	2866.0	5807.0	0.0	12829.0	0.0	
2017Q2	1236.0	3466.0	678.0	1326.0	4219.0	3756.0	0.0	159
2017Q3	0.0	4823.0	0.0	0.0	5608.0	0.0	0.0	330
2017Q4	0.0	4474.0	0.0	0.0	5574.0	0.0	0.0	243

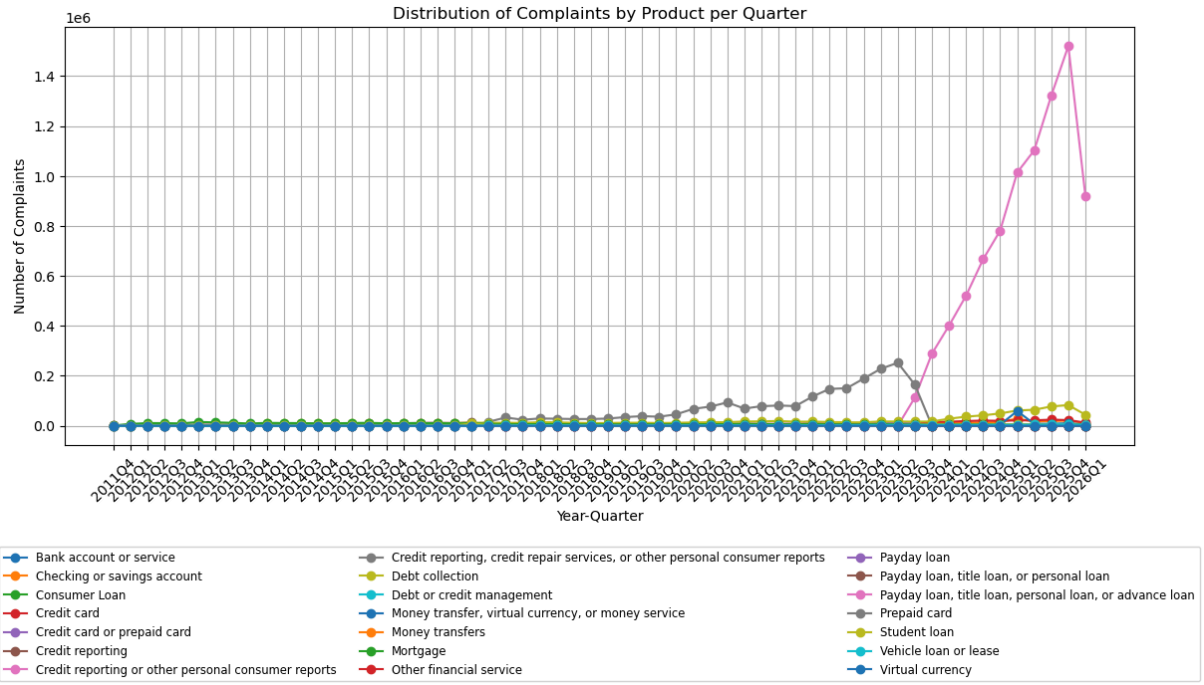
Product	Bank account or service	Checking or savings account	Consumer Loan	Credit card	Credit card or prepaid card	Credit reporting	Credit reporting or other personal consumer reports	C repor c re serv or o pers consu rep
year_quarter								
2018Q1	0.0	5112.0	0.0	0.0	6206.0	0.0	0.0	291
2018Q2	0.0	5497.0	0.0	0.0	6324.0	0.0	0.0	283
2018Q3	0.0	5178.0	0.0	0.0	5933.0	0.0	0.0	269
2018Q4	0.0	5420.0	0.0	0.0	5780.0	0.0	0.0	271
2019Q1	0.0	5238.0	0.0	0.0	6019.0	0.0	0.0	288
2019Q2	0.0	5507.0	0.0	0.0	6445.0	0.0	0.0	347
2019Q3	0.0	5664.0	0.0	0.0	6682.0	0.0	0.0	386
2019Q4	0.0	5326.0	0.0	0.0	6682.0	0.0	0.0	363
2020Q1	0.0	5252.0	0.0	0.0	7135.0	0.0	0.0	456
2020Q2	0.0	6314.0	0.0	0.0	9457.0	0.0	0.0	677
2020Q3	0.0	6712.0	0.0	0.0	8856.0	0.0	0.0	767
2020Q4	0.0	5960.0	0.0	0.0	8433.0	0.0	0.0	934
2021Q1	0.0	7287.0	0.0	0.0	8618.0	0.0	0.0	689
2021Q2	0.0	7475.0	0.0	0.0	7236.0	0.0	0.0	784
2021Q3	0.0	7173.0	0.0	0.0	7633.0	0.0	0.0	811
2021Q4	0.0	7621.0	0.0	0.0	8336.0	0.0	0.0	789
2022Q1	0.0	9320.0	0.0	0.0	9648.0	0.0	0.0	1172
2022Q2	0.0	8885.0	0.0	0.0	8950.0	0.0	0.0	1476
2022Q3	0.0	9675.0	0.0	0.0	10675.0	0.0	0.0	1502
2022Q4	0.0	9704.0	0.0	0.0	10608.0	0.0	0.0	1890
2023Q1	0.0	14638.0	0.0	0.0	12847.0	0.0	0.0	2283
2023Q2	0.0	11444.0	0.0	0.0	12679.0	0.0	0.0	2521
2023Q3	0.0	12405.0	0.0	6133.0	9785.0	0.0	113258.0	1643
2023Q4	0.0	11389.0	0.0	15740.0	0.0	0.0	288806.0	
2024Q1	0.0	12122.0	0.0	16322.0	0.0	0.0	400578.0	

Product	Bank account or service	Checking or savings account	Consumer Loan	Credit card	Credit card or prepaid card	Credit reporting	Credit reporting or other personal consumer reports	C repor c re serv or o pers consu rep
year_quarter								
2024Q2	0.0	12759.0	0.0	18988.0	0.0	0.0	520720.0	
2024Q3	0.0	13281.0	0.0	20757.0	0.0	0.0	666241.0	
2024Q4	0.0	14672.0	0.0	19978.0	0.0	0.0	781150.0	
2025Q1	0.0	28890.0	0.0	21541.0	0.0	0.0	1013732.0	
2025Q2	0.0	16180.0	0.0	21777.0	0.0	0.0	1102561.0	
2025Q3	0.0	19051.0	0.0	24532.0	0.0	0.0	1321435.0	
2025Q4	0.0	20300.0	0.0	22734.0	0.0	0.0	1520547.0	
2026Q1	0.0	10776.0	0.0	12587.0	0.0	0.0	919703.0	

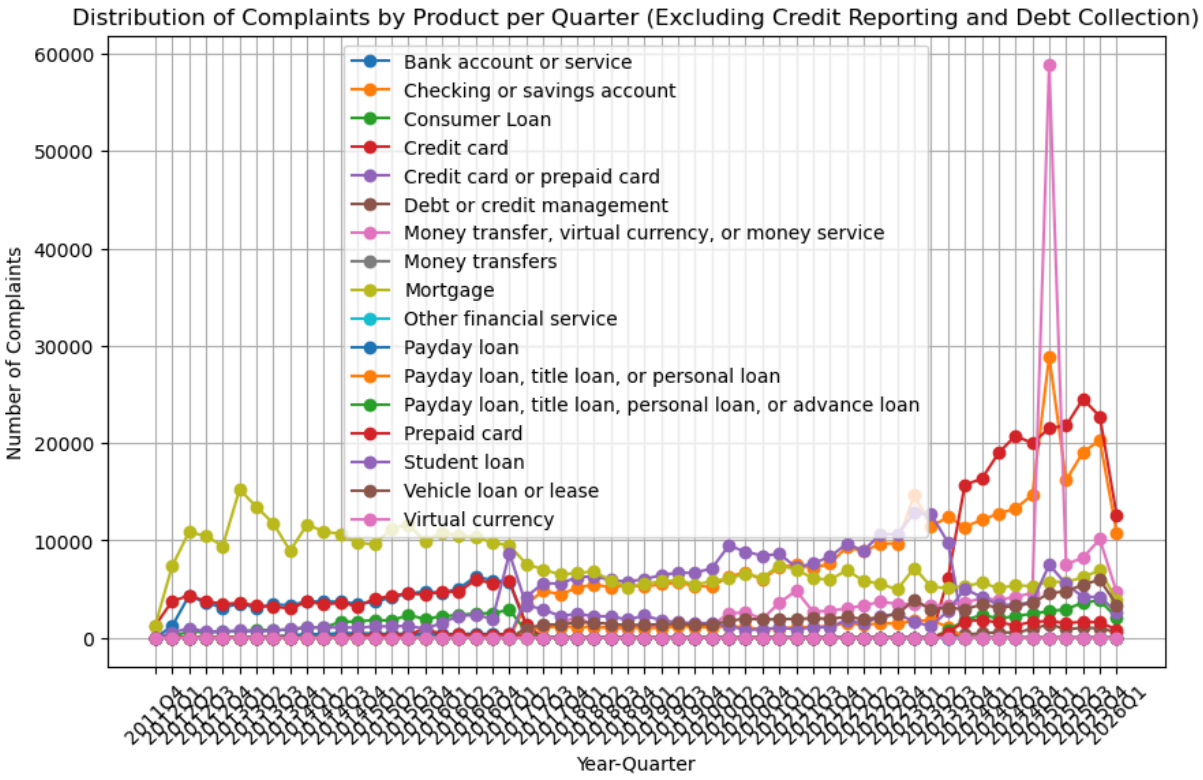
58 rows × 21 columns

In [32]:
```
# Plot a line chart for each product's complaint volume over time
plot_df = product_counts_qtr.copy()
plot_df['year_q_period'] = pd.PeriodIndex(plot_df['year_quarter'], freq='Q')

# Build one globally ordered quarter axis shared by all product series
plot_pivot = (
    plot_df.pivot_table(
        index='year_q_period',
        columns='Product',
        values='num_complaints',
        aggfunc='sum'
    )
    .fillna(0)
    .sort_index()
 )

x_labels = plot_pivot.index.astype(str)

plt.figure(figsize=(13, 7))
for product in plot_pivot.columns:
    plt.plot(x_labels, plot_pivot[product].values, marker='o', label=product)

plt.title('Distribution of Complaints by Product per Quarter')
plt.xlabel('Year-Quarter')
plt.ylabel('Number of Complaints')
plt.xticks(rotation=45)
plt.grid(True)
```

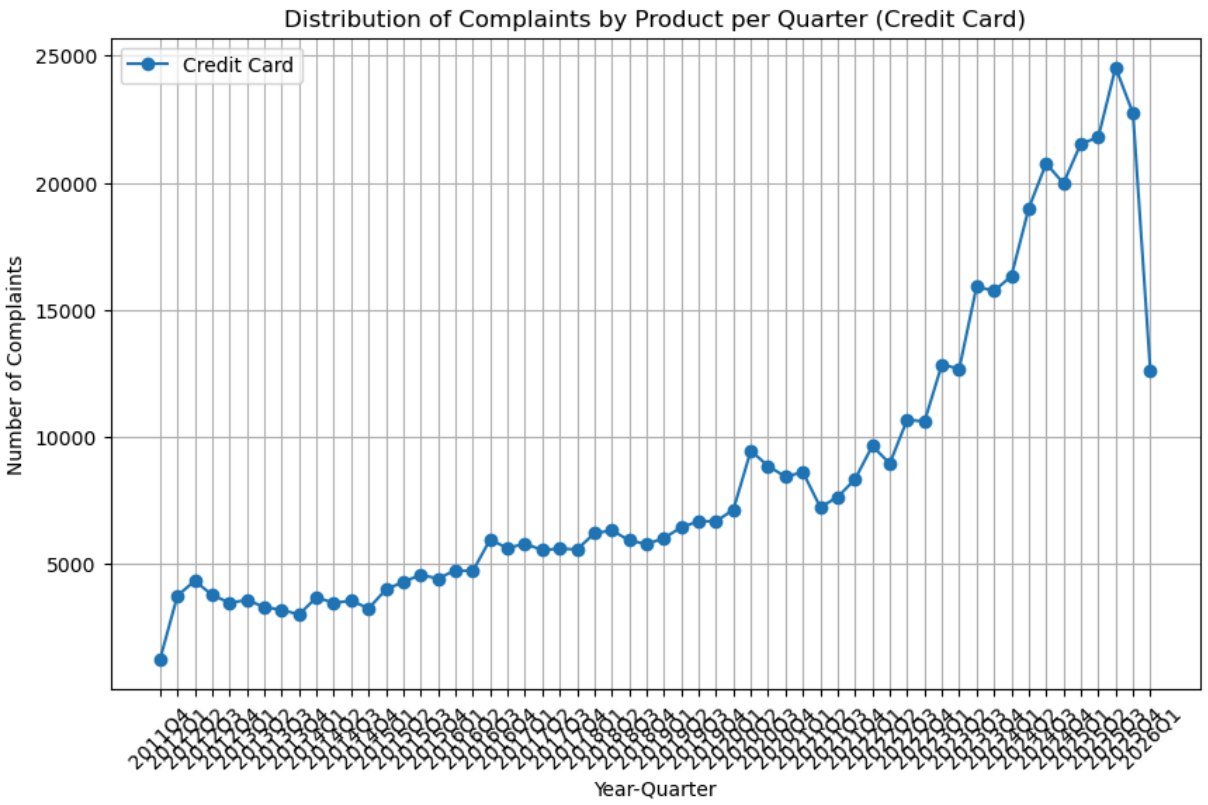
```
# Place legend below chart and spread across columns
plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.23), ncol=3, fontsize='small
plt.tight_layout()
plt.subplots_adjust(bottom=0.35)
plt.show()
```



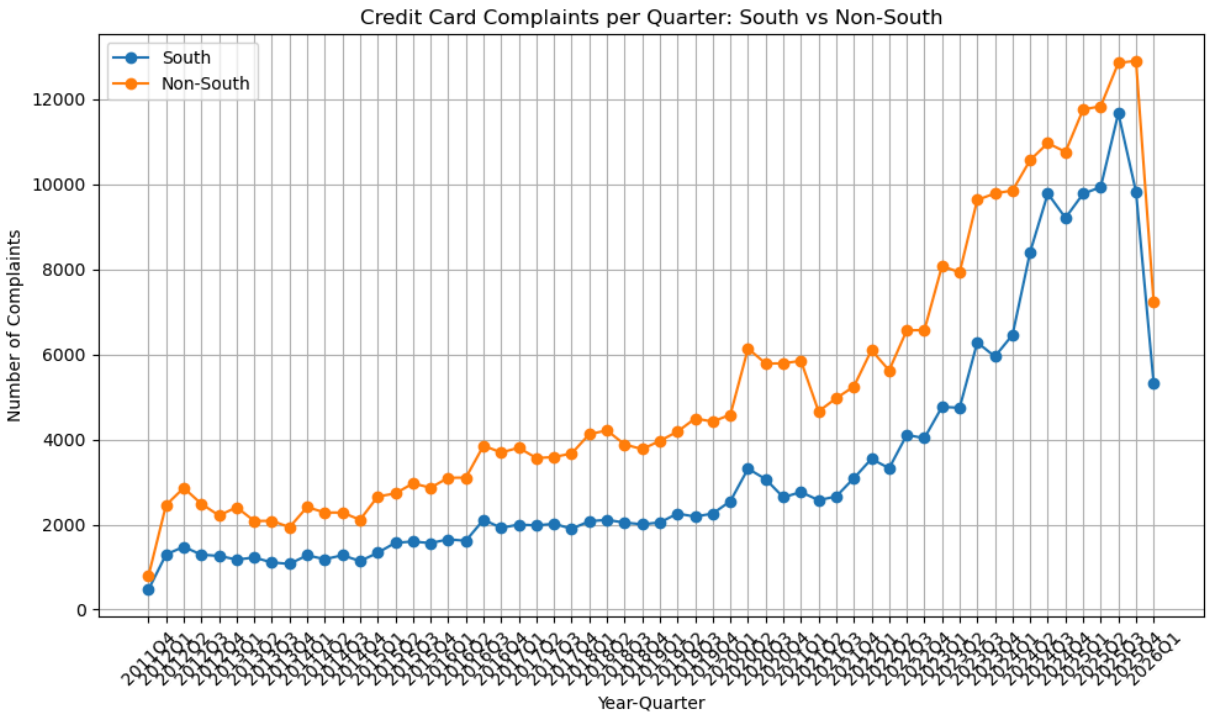
In [36]:
```
# Plotting the same chart but without the 3 categories starting with "credit report
plt.figure(figsize=(10, 6))
for product in plot_pivot.columns:
    if not product.lower().startswith('credit reporting') and not product.lower().s
        plt.plot(x_labels, plot_pivot[product].values, marker='o', label=product)

plt.title('Distribution of Complaints by Product per Quarter (Excluding Credit Repo
plt.xlabel('Year-Quarter')
plt.ylabel('Number of Complaints')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.show()
```



In [38]:
```
# Plot the same chart but only for the 2 categories starting with "credit card", co
plt.figure(figsize=(10, 6))
credit_card_values = plot_pivot[[col for col in plot_pivot.columns if col.lower().s
plt.plot(x_labels, credit_card_values, marker='o', label='Credit Card')

plt.title('Distribution of Complaints by Product per Quarter (Credit Card)')
plt.xlabel('Year-Quarter')
plt.ylabel('Number of Complaints')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.show()
```



In [41]:
```
# Plot the above (i.e., for the 2 categories starting with "credit card", combined
plt.figure(figsize=(10, 6))

credit_card_cols = [col for col in df['Product'].unique() if col.lower().startswith

for region_label in ['South', 'Non-South']:
    if region_label == 'South':
        region_df = df[df['region'] == 'South']
    else:
        region_df = df[df['region'] != 'South']

    region_counts = (
        region_df[region_df['Product'].isin(credit_card_cols)]
        .groupby('year_quarter')
        .size()
        .reset_index(name='num_complaints')
    )
    region_counts['year_q_period'] = pd.PeriodIndex(region_counts['year_quarter'],
    region_counts = region_counts.set_index('year_q_period').reindex(plot_pivot.ind

    plt.plot(x_labels, region_counts['num_complaints'].values, marker='o', label=re

plt.title('Credit Card Complaints per Quarter: South vs Non-South')
plt.xlabel('Year-Quarter')
plt.ylabel('Number of Complaints')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



Credit Card Subset Construction

After the broader product review, the analysis scope is restricted to records whose `Product` begins with `Credit card`, and that subset is then trimmed to the working date window used in notebooks 02 and 03.

In [ ]:
```
# Isolate complaints whose product label starts with "credit card"
df_creditcard = df[df['Product'].str.lower().str.startswith('credit card')]
df_creditcard['Product'].value_counts()
```

Out[ ]:
```
Product
Credit card                     290278
Credit card or prepaid card     206368
Name: count, dtype: int64
```

In [43]:
```
df_creditcard
```

Out[43]:

	Date received	Product	Sub-product	Issue	Sub-issue	Consumer complaint narrative	Compa pub respor
16	2020-03-09	Credit card or prepaid card	General-purpose credit card or charge card	Fees or interest	Problem with fees	On XX/XX/2020 I booked a hotel in XXXX that re...	N
29	2020-03-18	Credit card or prepaid card	General-purpose credit card or charge card	Fees or interest	Problem with fees	I have had a Barclays/Uber credit card for nea...	Compa l respond to consun and th
114	2023-12-15	Credit card	General-purpose credit card or charge card	Problem with a company's investigation into an...	Was not notified of investigation status or re...	Take a look at the attached documents. I reque...	Compa l respond to consun and th
141	2020-01-04	Credit card or prepaid card	Government benefit card	Trouble using the card	Trouble getting information about the card	I received my new card for my child support an...	Compa l respond to consun and th
142	2019-07-31	Credit card or prepaid card	General-purpose credit card or charge card	Fees or interest	Problem with fees	NaN	Compa l respond to consun and th
...	...	...	...	...	...	...	
13843530	2013-11-02	Credit card	NaN	Other fee	NaN	NaN	N
13843536	2013-04-04	Credit card	NaN	Credit reporting	NaN	NaN	N
13843547	2013-06-21	Credit card	NaN	Payoff process	NaN	NaN	N
13843548	2023-06-30	Credit card or prepaid card	General-purpose credit card or charge card	Other features, terms, or problems	Other problem	NaN	N

	Date received	Product	Sub-product	Issue	Sub-issue	Consumer complaint narrative	Compa pub respor
13843555	2014-04-21	Credit card	NaN	Other	NaN	NaN	N

496646 rows × 22 columns

In [11]:
```
# Dates range for the credit card subset
print(f"Dates range from {min(df_creditcard['Date received'])} to {max(df_creditcar
```

```
Dates range from 2011-12-01 00:00:00 to 2026-02-24 00:00:00
```

In [ ]:
```
# Restrict the credit card subset to the working analysis window
df_creditcard = df_creditcard[
    (df_creditcard['Date received'] >= '2012-01-01') &
    (df_creditcard['Date received'] <= '2025-12-31')
]
```

In [13]:
```
# Dates range for the credit card subset
print(f"Dates range from {min(df_creditcard['Date received'])} to {max(df_creditcar
```

```
Dates range from 2012-01-01 00:00:00 to 2025-12-31 00:00:00
```

Conclusion

This notebook establishes the data-preparation pipeline that now supports the credit card-focused analysis track.

- Time and region features are engineered on the raw CFPB complaints data so the same structure can be reused across downstream notebooks.
- Product-level diagnostics show why it is reasonable to isolate the credit card portfolio instead of carrying every product into later analysis.
- The exported artifact is a trimmed credit card subset, bounded to `2012-01-01` through `2025-12-31`, ready for notebook 02 EDA and the notebook 03 forecasting workflows.

In [ ]:
```
# Export the trimmed credit card subset used by notebooks 02 and 03
df_creditcard.to_parquet(
    "../data/processed/credit_card_dataset.parquet",
    index=False
)
```